Web Technologies

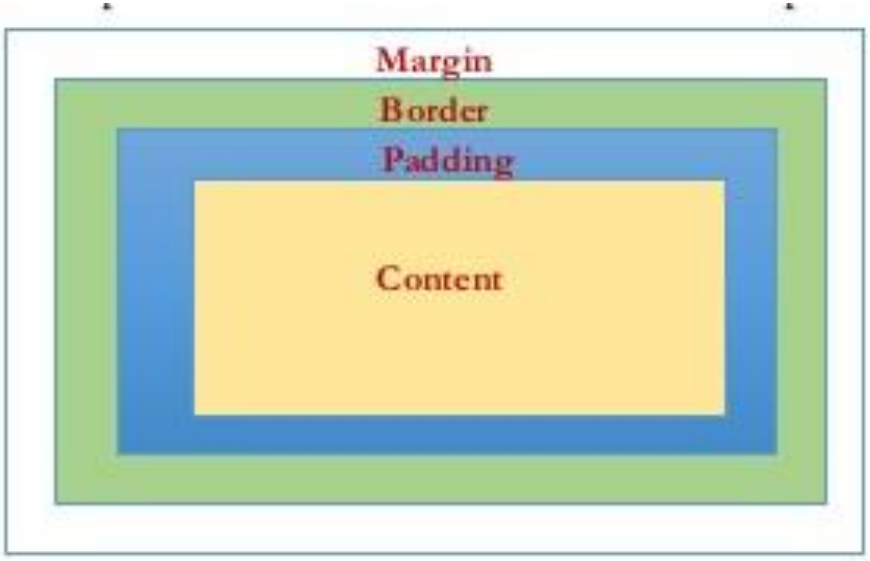
CSS Properties

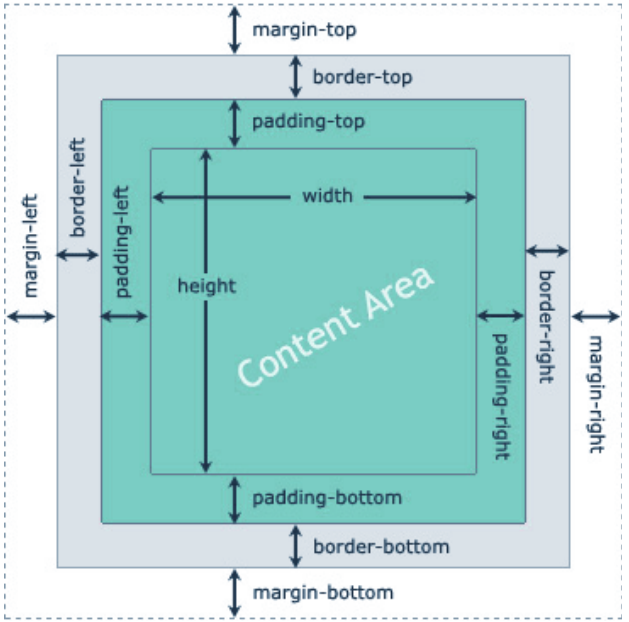
Summary of Today's Lecture

- CSS Box Model
  - Height and Width
  - Padding
  - Borders
  - Margins
- CSS Box Sizing
- CSS Text
- CSS Fonts
- CSS Colors
- CSS Backgrounds
- CSS Links
- CSS Lists
- CSS Tables
- CSS Display
- CSS Float
- CSS Position
- CSS Opacity
- CSS Icons

CSS Box Model

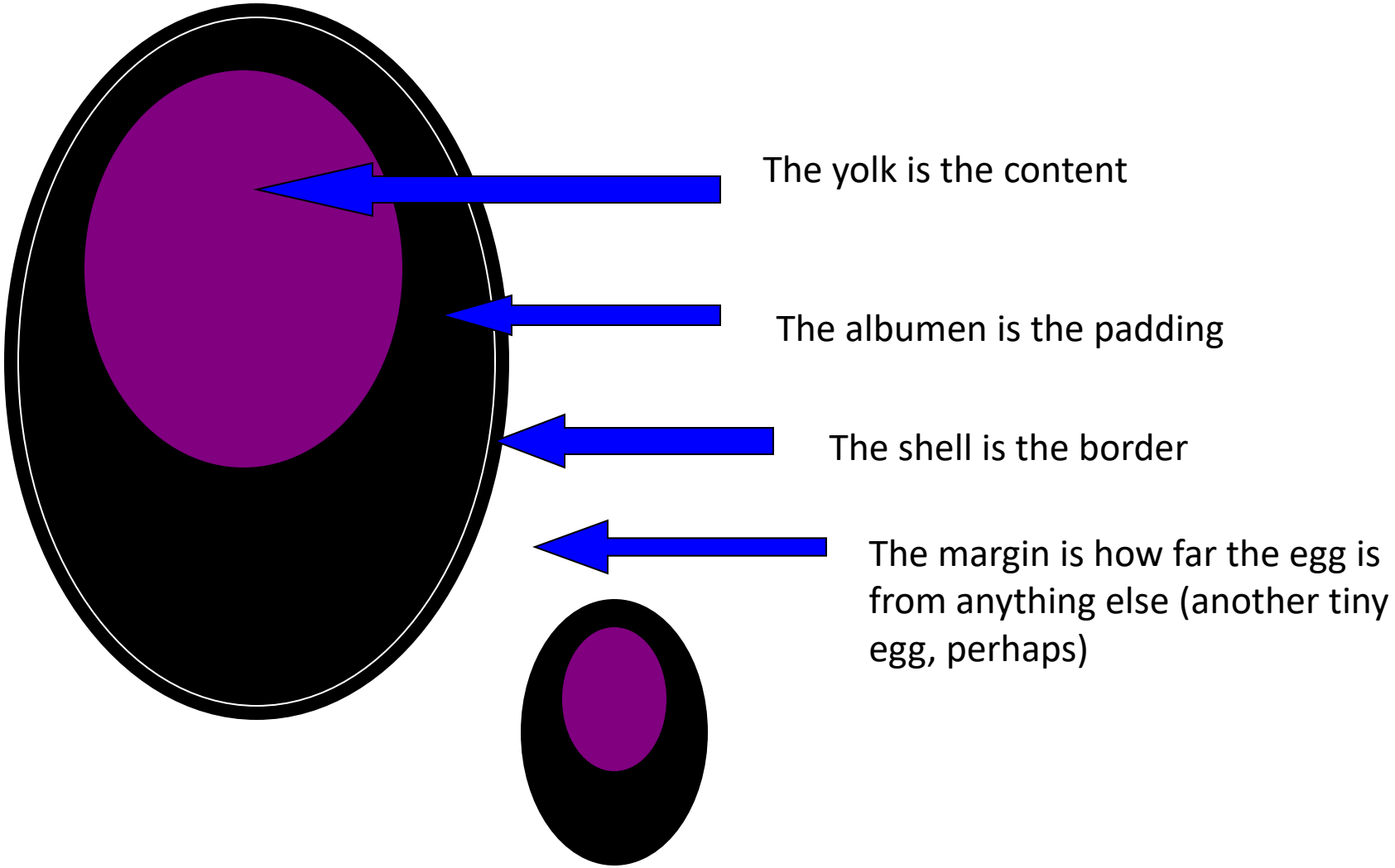
- All HTML elements can be considered as boxes. In CSS, the term "box model" is used when talking about design and layout.
- The CSS box model is essentially a box that wraps around every HTML element. It consists of `margins`, `borders`, `padding`, and the actual `content`.
  - **Content** - The content of the box, where text and images appear.
  - **Padding** - Clears an area around the content. The padding is transparent.
  - **Border** - A border that goes around the padding and content.
  - **Margin** - Clears an area outside the border. The margin is transparent.





CSS Box Model

Think of it like an egg:



CSS Height and Width

- The *height* and *width* properties are used to set the height and width of an element.
- The height and width properties do not include padding, borders, or margins. It sets the height/width of the area inside the padding, border, and margin of the element.
- The height and width properties may have the following values:
  - auto - This is default. The browser calculates the height and width
  - length - Defines the height/width in px, cm etc.
  - % - Defines the height/width in percent of the containing block
  - initial - Sets the height/width to its default value
  - inherit - The height/width will be inherited from its parent value
- Inline elements expand and contract horizontally and vertically to accommodate their content. The default height of an element is determined by its content.
- Inline-level elements cannot have a fixed size; thus the width and height properties are only relevant to non-inline elements.
- The CSS *max-width* property is used to set the maximum width of an element.

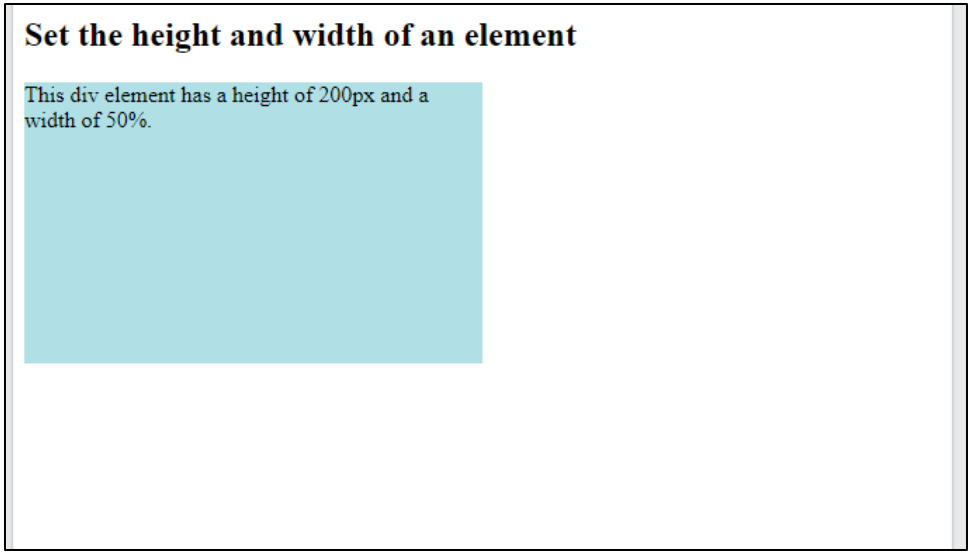
CSS Height and Width Example

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  height: 200px;
  width: 50%;
  border: 1px solid black;
}
</style>
</head>
<body>

<h2>Set the height and width of an element</h2>

<p>This div element has a height of 200px and a width of 50%:</p>

<div></div>
```



CSS Padding

- The padding property can be used to specify space between element content and element border.
- With CSS, you have full control over the padding. There are properties for setting the padding for each side of an element (top, right, bottom, and left).
- CSS has properties for specifying the padding for each side of an element:
  - padding-top
  - padding-right
  - padding-bottom
  - padding-left
  - ```
    div {
      padding-top: 50px;
      padding-right: 30px;
      padding-bottom: 50px;
      padding-left: 80px;
    }
    ```
- All the padding properties can have the following values:
  - length - specifies a padding in px, pt, cm, etc.
  - % - specifies a padding in % of the width of the containing element
  - inherit - specifies that the padding should be inherited from the parent element
    ```

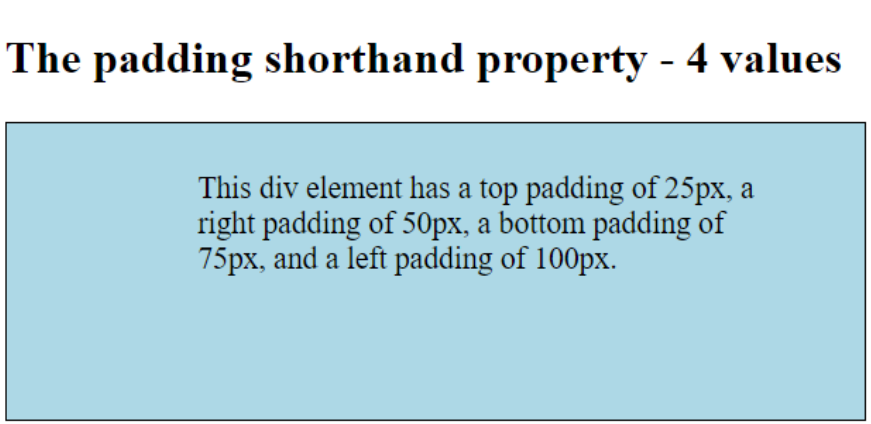
CSS Padding

- To shorten the code, it is possible to specify all the padding properties in one property.
- The `padding` property is a shorthand property for the following individual padding properties:
  - padding-top
  - padding-right
  - padding-bottom
  - padding-left
- If the padding property has four values:
  - padding: 25px 50px 75px 100px;
    - top padding is 25px
    - right padding is 50px
    - bottom padding is 75px
    - left padding is 100px

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 300px;
  border: 1px solid black;
  padding: 25px 50px 75px 100px;
}
</style>
</head>
<body>

<h2>The padding shorthand property - 4 values</h2>

<div>This div element has a top padding of 25px, a
right padding of 50px, a bottom padding of 75px, and
a left padding of 100px.</div>
```



CSS Margins

- The margin property allows us to set the amount of space that surrounds an element.
- The CSS margin properties are used to create space around elements, outside of any defined borders.
- With CSS, you have full control over the margins. There are properties for setting the margin for each side of an element (top, right, bottom, and left).
  - margin-top
  - margin-right
  - margin-bottom
  - margin-left
  - ```
    p {
      margin-top: 100px;
      margin-bottom: 100px;
      margin-right: 150px;
      margin-left: 80px;
    }
    ```
- All the margin properties can have the following values:
  - auto - the browser calculates the margin
  - length - specifies a margin in px, pt, cm, etc.
  - % - specifies a margin in % of the width of the containing element
  - inherit - specifies that the margin should be inherited from the parent element
    ```

CSS Margins

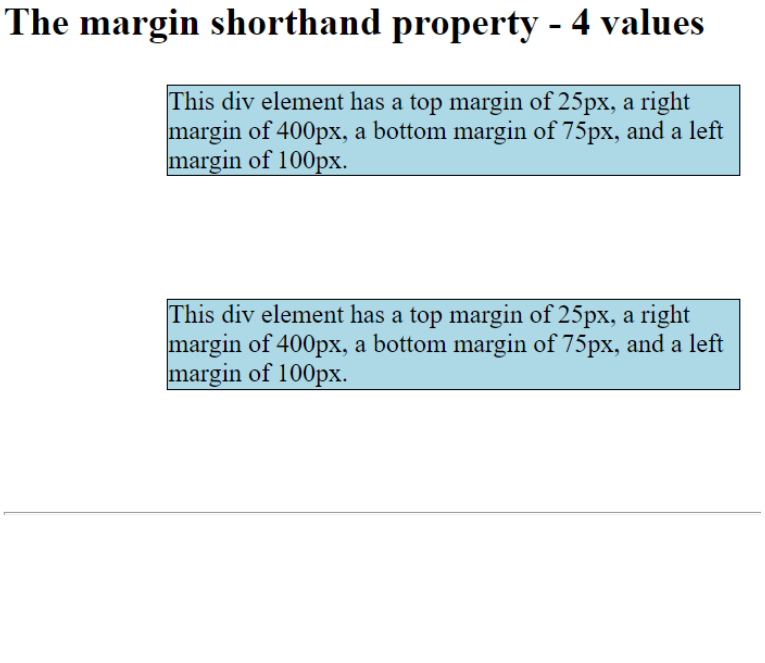
- To shorten the code, it is possible to specify all the padding properties in one property.
- The `margin` property is a shorthand property for the following individual padding properties:
  - margin-top
  - margin-right
  - margin-bottom
  - margin-left
- If the margin property has four values:
  - margin: 25px 400px 75px 100px;
    - top margin is 25px
    - right margin is 400px
    - bottom margin is 75px
    - left margin is 100px`

```
<style>
div {
  border: 1px solid black;
  margin: 25px 400px 75px 100px;
  background-color: lightblue;
}
</style>
</head>
<body>

<h2>The margin shorthand property - 4
values</h2>

<div>This div element has a top margin of
25px, a right margin of 400px, a bottom
margin of 75px, and a left margin of
100px.</div>

<div>This div element has a top margin of
25px, a right margin of 400px, a bottom
margin of 75px, and a left margin of
100px.</div>
```



CSS Padding/ Margins for Inline Elements

- Inline-level elements are affected a bit differently than block and inline-block elements when it comes to margins and padding.
  - Margins only work horizontally—left and right—on inline-level elements.
  - Padding works on all four sides of inline-level elements; however, the vertical padding—the top and bottom—may bleed into the lines above and below an element.

CSS Borders (Style)

- Borders fall between the padding and margin, providing an outline around an element.
- The border property requires three values: **width**, **style**, and **color**.
- The border-style property specifies what kind of border to display. The following values are allowed:
  - **dotted** - Defines a dotted border
  - **dashed** - Defines a dashed border
  - **solid** - Defines a solid border
  - **double** - Defines a double border
  - **groove** - Defines a 3D grooved border. The effect depends on the border-color value
  - **ridge** - Defines a 3D ridged border. The effect depends on the border-color value
  - **inset** - Defines a 3D inset border. The effect depends on the border-color value
  - **outset** - Defines a 3D outset border. The effect depends on the border-color value
  - **none** - Defines no border
  - hidden - Defines a hidden border
- The `border-style` property can have from one to four values (for the top border, right border, bottom border, and the left border).

CSS Borders (Width)

- The `border-width` property specifies the width of the four borders.
- The width can be set as a specific size (in px, pt, cm, em, etc) or by using one of the three pre-defined values: thin, medium, or thick.
- The border-width property can have from one to four values (for the top border, right border, bottom border, and the left border):

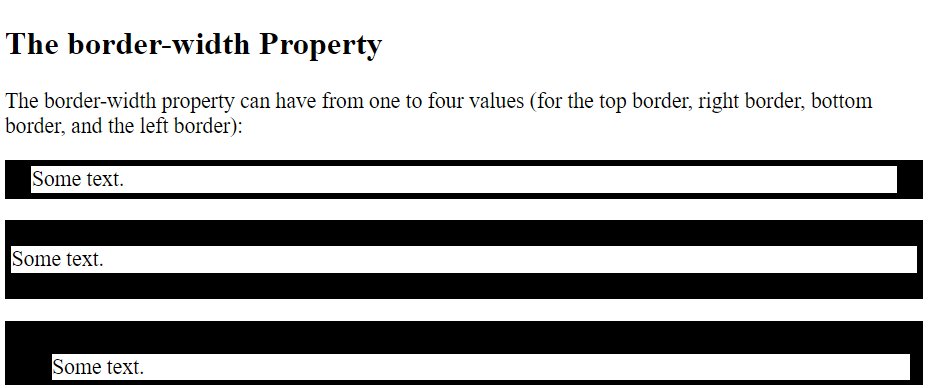
```
<html>
<head>
<style>
p.one {
  border-style: solid;
  border-width: 5px 20px; /* 5px top and bottom, 20px on the sides */
}

p.two {
  border-style: solid;
  border-width: 20px 5px; /* 20px top and bottom, 5px on the sides */
}

p.three {
  border-style: solid;
  border-width: 25px 10px 4px 35px; /* 25px top, 10px right, 4px bottom and 35px
left */
}
</style>
</head>
<body>

<h2>The border-width Property</h2>
<p>The border-width property can have from one to four values (for the top
border, right border, bottom border, and the left border):</p>

<p class="one">Some text.</p>
<p class="two">Some text.</p>
<p class="three">Some text.</p>
```



Activate Windows
Go to Settings to activate Windows.

CSS Borders (Color)

- The border-color property is used to set the color of the four borders.
- The color can be set by:
  - name - specify a color name, like "red"
  - HEX - specify a HEX value, like "#ff0000"
  - RGB - specify a RGB value, like "rgb(255,0,0)"

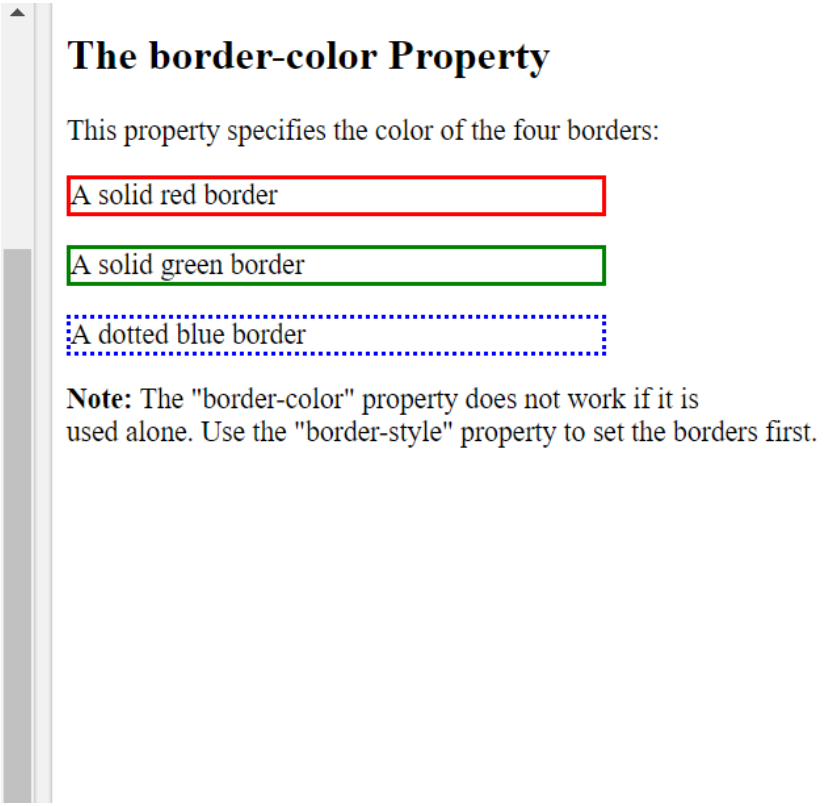
```
p.two {
  width: 300px;
  border-style: solid;
  border-color: green;
}

p.three {
  width: 300px;
  border-style: dotted;
  border-color: blue;
}
</style>
</head>
<body>

<h2>The border-color Property</h2>
<p>This property specifies the color of the four borders:</p>

<p class="one">A solid red border</p>
<p class="two">A solid green border</p>
<p class="three">A dotted blue border</p>

<p><b>Note:</b> The "border-color" property does not work if
it is <br> used alone. Use the "border-style" property to set
the borders first.</p>
```



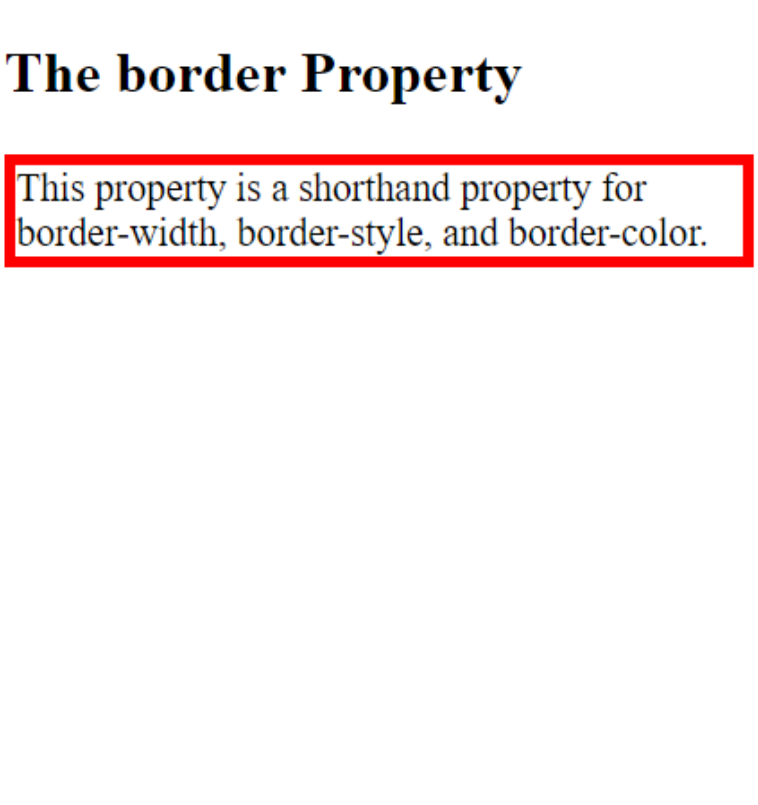
CSS Borders (Shorthand)

- To shorten the code, it is also possible to specify all the individual border properties in one property.
- The border property is a shorthand property for the following individual border properties:
  - border-width
  - border-style (required)
  - border-color

```
<!DOCTYPE html>
<html>
<head>
<style>
p {
  width:300px;
  border: 5px solid red;
}
</style>
</head>
<body>

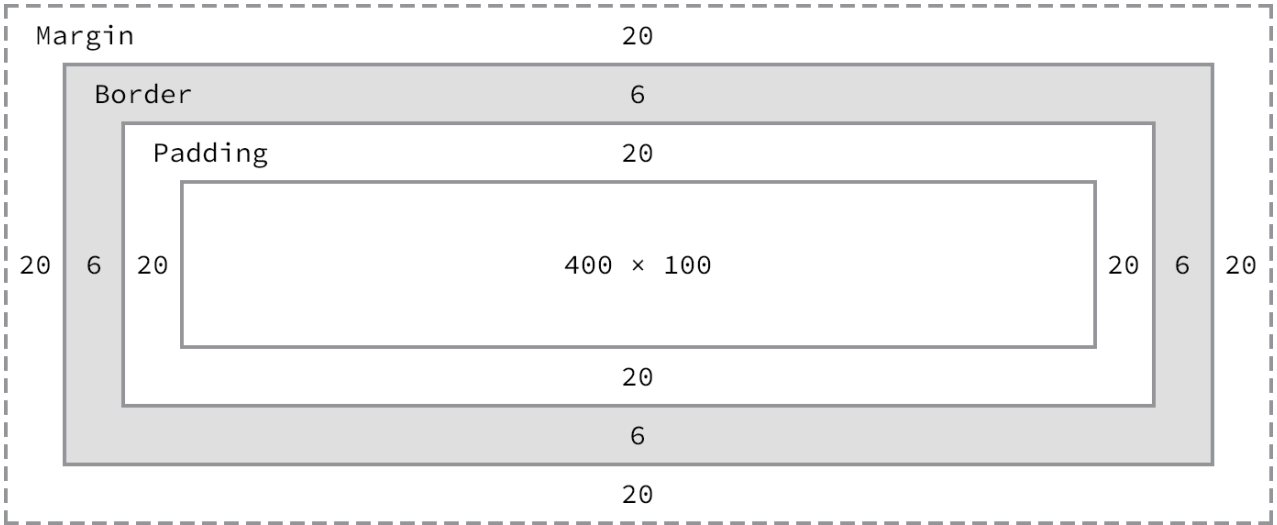
<h2>The border Property</h2>

<p>This property is a shorthand
property for border-width,
border-style, and border-color.
</p>
```



Size of a Box

- According to the box model, the total width of an element can be calculated using the following formula:
  - `margin-right + border-right + padding-right + width + padding-left + border-left + margin-left`
- the total height of an element can be calculated using the following formula:
  - `margin-top + border-top + padding-top + height + padding-bottom + border-bottom + margin-bottom`
- Using the formulas, we can find the total height and width of our example code.
  - **Width:** `492px = 20px + 6px + 20px + 400px + 20px + 6px + 20px`
  - **Height:** `192px = 20px + 6px + 20px + 100px + 20px + 6px + 20px`



Box-Sizing

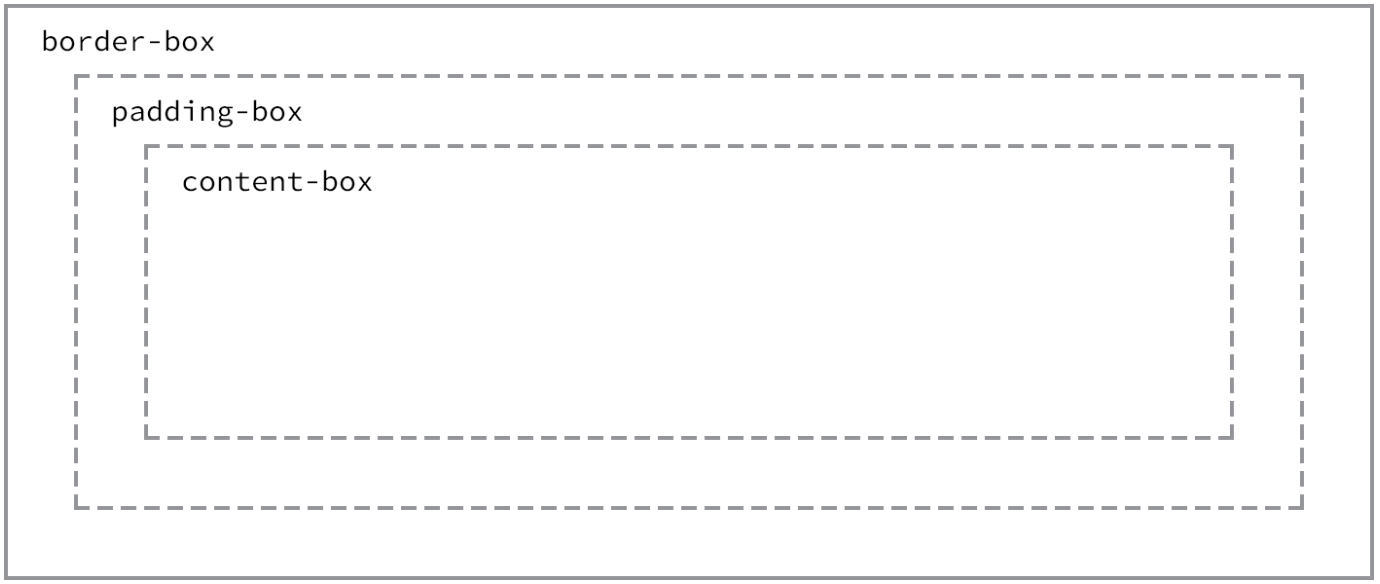
- CSS3 introduced the box-sizing property, which allows us to change exactly how the box model works and how an element's size is calculated. The property accepts two primary values:
  - content-box,
  - border-box
- Each of these values has a slightly different impact on how the box size is calculated.

Content Box

- The content-box value is the default value, leaving the box model as an additive design. If we don't use the box-sizing property, this will be the default value for all elements.
- The size of an element begins with the width and height properties, and then any padding, border, or margin property values are added.
  - div {
    - -webkit-box-sizing: content-box;
    - -moz-box-sizing: content-box;
    - box-sizing: content-box; }
  - Mozilla Firefox: -moz-
  - Microsoft Internet Explorer: -ms-
  - Webkit (Google Chrome and Apple Safari): -webkit-

Border-Box

- The border-box value alters the box model so that any border or padding property values are included within the width and height of an element.
- When using the border-box value, if an element has a width of 400 pixels, a padding of 20 pixels around every side, and a border of 10 pixels around every side, the actual width will remain 400 pixels.
- If we add a margin, those values will need to be added to calculate the full box size. No matter which box-sizing property value is used, any margin values will need to be added to calculate the full size of the element.



CSS Fonts

- **Font Family**
  - The font family of a text is set with the font-family property. In CSS, there are two types of font family names:
    - generic family - a group of font families with a similar look (like "Serif" or "Monospace")
    - font family - a specific font family (like "Times New Roman" or "Arial").
  - The font-family property should hold several font names as a "fallback" system. If the browser does not support the first font, it tries the next font, and so on.
  - Start with the font you want, and end with a generic family, to let the browser pick a similar font in the generic family, if no other fonts are available.
  - More than one font family is specified in a comma-separated list.
    - **Syntax:** `p { font-family: "Times New Roman", Arial, serif; }`
- **Font Style**
  - The font-style property is mostly used to specify italic text.
    - **Syntax:** `p.italic { font-style: italic; }`
- **Font Weight**
  - The font-weight property specifies the weight of a font:
    - **Syntax:** `p.thick { font-weight: bold;}`

CSS Fonts

- **Font Size**
  - The font-size property sets the size of the text. The font-size value can be an absolute, or relative size.
  - Absolute size (e.g. px etc.):
    - Sets the text to a specified size
    - Does not allow a user to change the text size in all browsers (bad for accessibility reasons)
    - Absolute size is useful when the physical size of the output is known
  - Relative size (e.g. em etc.):
    - Sets the size relative to surrounding elements
    - Allows a user to change the text size in browsers
  - **Syntax:** `h1 { font-size: 40px; }`
- **Font Google**
  - If you do not want to use any of the standard fonts in HTML, you can use the Google Fonts API to add hundreds of other fonts to your page.
  - Just add a stylesheet link and refer to a font family of your choice:
    - ```
      <head>
      <link rel="stylesheet" href="https://fonts.googleapis.com
      /css?family=Sofia">
      ```
    - ```
      </head>
      ```

CSS Fonts

- **Font Shorthand**
  - To shorten the code, it is also possible to specify all the individual font properties in one property.
  - The `font` property is a shorthand property for:
    - font-style
    - font-variant
    - font-weight
    - font-size/line-height
    - font-family
- **Syntax:**
```
p.a { font: 20px Arial, sans-serif; }
}
```

All CSS Font Properties

Property	Description
font	Sets all the font properties in one declaration
font-family	Specifies the font family for text
font-size	Specifies the font size of text
font-style	Specifies the font style for text
font-variant	Specifies whether or not a text should be displayed in a small-caps font
font-weight	Specifies the weight of a font

CSS Text

- **Text Color**
  - The `color` property is used to set the color of the text. The color is specified by:
    - a color name - like "red"
    - a HEX value - like "#ff0000"
    - an RGB value - like "rgb(255,0,0)“
      - **Syntax:** `p { color: green; }`
- **Text Alignment**
  - The `text-align` property is used to set the horizontal alignment of a text.
  - A text can be left or right aligned, centered, or justified.
    - **Syntax:** `p { text-align: center; }`
- **Text Decoration**
  - The `text-decoration` property is used to set or remove decorations from text. The possible values of this property are overline, underline etc.
    - **Syntax:** `h3 {text-decoration: underline; }`

CSS Text

- **Text Transformation**
  - The `text-transform` property is used to specify uppercase and lowercase letters in a text.
  - It can be used to turn everything into uppercase or lowercase letters or capitalize the first letter of each word.
    - **Syntax:** `p.lowercase {text-transform: lowercase;}`
- **Text Spacing**
  - Text Indentation
  - Letter Spacing
  - Line Height
  - Word Spacing
  - White Space.
- **Text Shadow**
  - The `text-shadow` property adds shadow to text.
  - In its simplest use, you only specify the horizontal shadow (2px) and the vertical shadow (2px):
    - **Syntax:** `h1 {text-shadow: 2px 2px;}`

CSS Backgrounds

- The CSS background properties are used to define the background effects for elements.
- background-color
  - The background-color property specifies the background color of an element.
  - With CSS, a color is most often specified by:
    - a valid color name - like "red"
    - a HEX value - like "#ff0000"
    - an RGB value - like "rgb(255,0,0)"
      - **Syntax:** body { background-color: lightblue; }
- background-image
  - The background-image property specifies an image to use as the background of an element.
  - By default, the image is repeated so it covers the entire element.
    - **Syntax:** body { background-image: url("paper.gif"); }

CSS Backgrounds

- background-repeat
  - By default, the background-image property repeats an image both horizontally and vertically.
  - Some images should be repeated only horizontally or vertically, or they will look strange, like this:
    - `body { background-image: url("gradient_bg.png"); }`
  - Showing the background image only once is also specified by the background-repeat property:
    - `body { background-image: url("img_tree.png"); background-repeat: no-repeat; }`
- background-position
  - The background-position property is used to specify the position of the background image.
    - `body { background-image: url("img_tree.png"); background-repeat: no-repeat; background-position: right top; }`

CSS Backgrounds

- background-attachment
  - The background-attachment property specifies whether the background image should scroll or be fixed (will not scroll with the rest of the page):
    - ```
      body { background-image: url("img_tree.png");
        background-repeat: no-repeat;
        background-position: right top;
        background-attachment: fixed; }
      ```
- Background Shorthand
  - To shorten the code, it is also possible to specify all the background properties in one single property. This is called a shorthand property.
  - When using the shorthand property the order of the property values is:
    - background-color
    - background-image
    - background-repeat
    - background-attachment
    - background-position
      - ```
        body { background: #ffffff url("img_tree.png") no-repeat right top; }
        ```

CSS Lists

- CSS provides the several properties for styling and formatting the most commonly used unordered and ordered lists. These CSS list properties typically allow you to:
- **Changing the Marker Type of Lists**
  - The list-style-type property specifies the type of list item marker.
  - The following example shows some of the available list item markers:

```
<head>
<style>
ul.a {
  list-style-type: circle;
}

ul.b {
  list-style-type: square;
}

ol.c {
  list-style-type: upper-roman;
}

ol.d {
  list-style-type: lower-alpha;
}
</style>
</head>
<body>
```

Example of unordered lists:

- Coffee
- Tea
- Coca Cola

- Coffee
- Tea
- Coca Cola

Example of ordered lists:

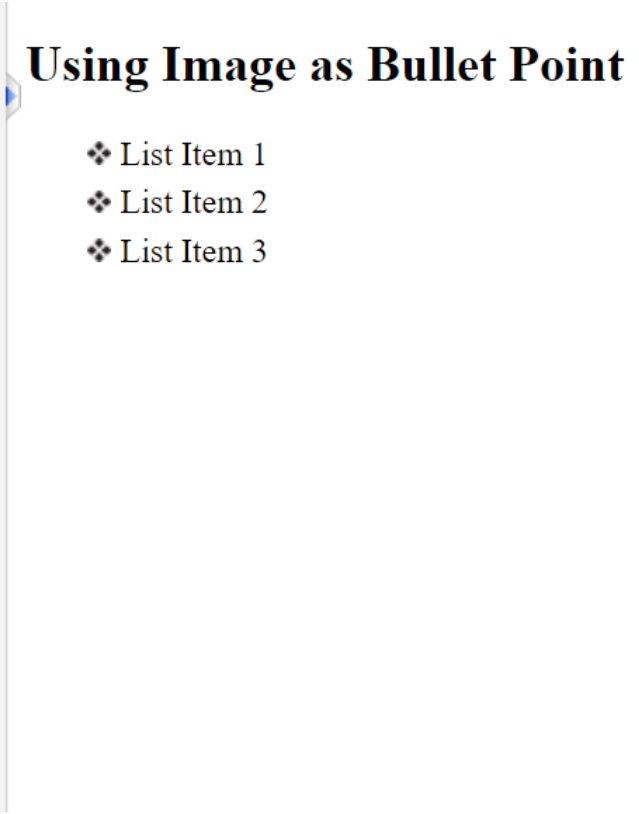
I. Coffee
II. Tea
III. Coca Cola

a. Coffee
b. Tea
c. Coca Cola

CSS Lists

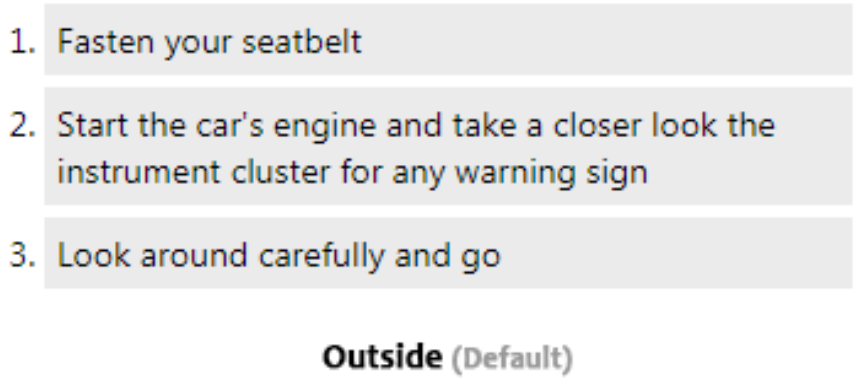
- **Using Images as List Markers**
  - The list-style-image property specifies an image as the list item marker.

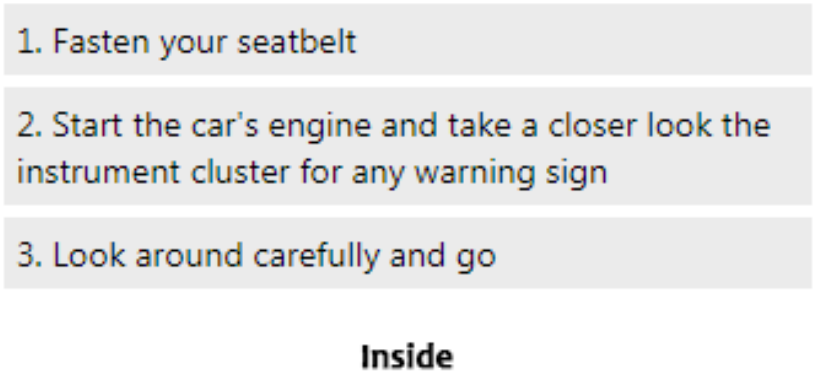
```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Setting an Image as List Marker with CSS</title>
<style>
    ul li {
        list-style-image: url("/examples/images/bullet.png");
        margin: 5px;
    }
</style>
</head>
<body>
    <h2>Using Image as Bullet Point</h2>
    <ul>
        <li>List Item 1</li>
        <li>List Item 2</li>
        <li>List Item 3</li>
    </ul>
</body>
</html>
```



CSS Lists

- **Changing the Position of List Markers**
  - The list-style-position property specifies the position of the list-item markers (bullet points).
    - "`list-style-position: outside;`" means that the bullet points will be outside the list item. The start of each line of a list item will be aligned vertically. This is default:
    - "`list-style-position: inside;`" means that the bullet points will be inside the list item. As it is part of the list item, it will be part of the text and push the text at the start:





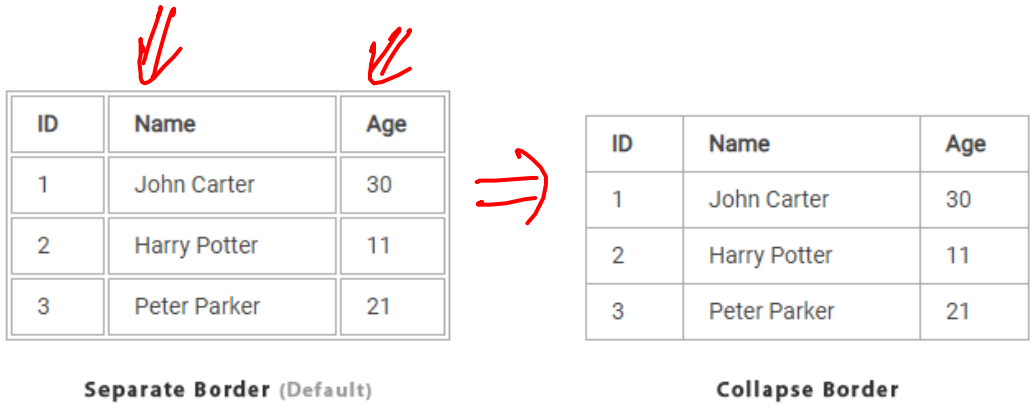
CSS Lists

- **List - Shorthand property**
  - The list-style property is a shorthand property. It is used to set all the list properties in one declaration:
  - When using the shorthand property, the order of the property values are:
    - **list-style-type** (if a list-style-image is specified, the value of this property will be displayed if the image for some reason cannot be displayed)
    - **list-style-position** (specifies whether the list-item markers should appear inside or outside the content flow)
    - **list-style-image** (specifies an image as the list item marker)
      - If one of the property values above are missing, the default value for the missing property will be inserted, if any.

```
» ul { list-style: square inside
  url("sqpurple.gif"); }
```

CSS Tables

- CSS provides several properties that allow you to control the layout and presentation of the table elements.
- **Adding Borders to Tables**
  - The CSS border property is the best way to define the borders for the tables.
  - The following example will set a black border for the <table>, <th>, and <td> elements.
    - `table, th, td { border: 1px solid black; }`
- By default, browser draws a border around the table, as well as around all the cells, with some space in-between, which results in double border. To get rid of this double border problem you can simply collapse the adjoining table cell borders and create clean single line borders.



ID	Name	Age
1	John Carter	30
2	Harry Potter	11
3	Peter Parker	21

Separate Border (Default)

ID	Name	Age
1	John Carter	30
2	Harry Potter	11
3	Peter Parker	21

Collapse Border

CSS Tables

- **Collapsing Table Borders**
  - There are two distinct models for setting borders on table cells in CSS: separate and collapse.
  - In the separate border model, which is the default, each table cell has its own distinct borders, whereas in the collapsed border model, adjacent table cells share a common border. You can set the border model for an HTML table by using the CSS border-collapse property.
  - The following style rules will collapse the table cell borders and apply one-pixel black border.
    - `table { border-collapse: collapse; }`
    - `th, td { border: 1px solid black; }`
- Table Width and Height
- Horizontal Alignment of Cell Contents
- Vertical Alignment of Cell Contents
- Table Padding
- Horizontal Dividers
- Hoverable Table
- Striped Tables
- Table Color

CSS Links

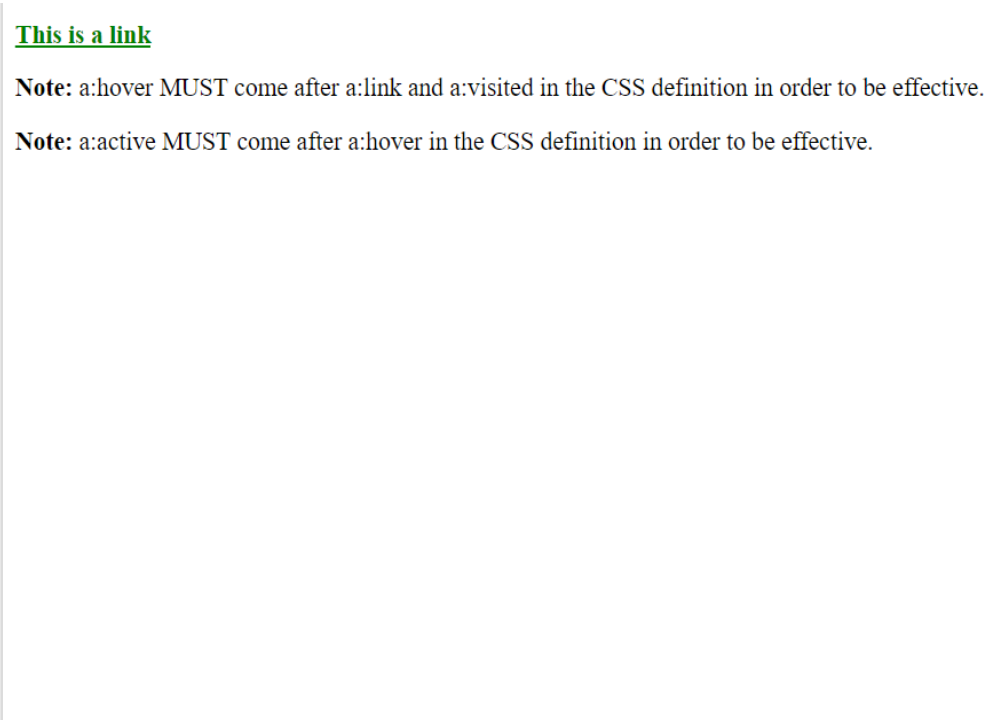
- Links can be styled with any CSS property (e.g. color, font-family, background, etc.).
- The four links states are:
  - a:link - a normal, unvisited link
  - a:visited - a link the user has visited
  - a:hover - a link when the user mouses over it
  - a:active - a link the moment it is clicked
- When setting the style for several link states, there are some order rules:
  - a:hover MUST come after a:link and a:visited
  - a:active MUST come after a:hover

```
<!DOCTYPE html>
<html>
<head>
<style>
/* unvisited link */
a:link {
  color: red;
}

/* visited link */
a:visited {
  color: green;
}

/* mouse over link */
a:hover {
  color: hotpink;
}

/* selected link */
a:active {
  color: blue;
}
</style>
</head>
<body>
```



CSS Display

- The CSS specification defines the default display value for all the elements, e.g. the element is rendered as block, while the element is displayed inline.
- **Display Block**
  - The block value of the display property forces an element to behave like block-level element, like a `<div>` or `<p>` element. The style rules in the following example displays the `<span>` and `<a>` elements as block-level elements:
    - `span { display: block; } a { display: block; }`

```
<!DOCTYPE html>
<html>
<head>
<style>
span {
  display: block;
  border: 1px solid black;
}
</style>
</head>
<body>

<span>A display property with a value of "block" results in</span>
<span>a line break between the two elements.</span>
```

A display property with a value of "block" results in
a line break between the two elements.

CSS Display

- **Display Inline**
  - The inline value of the display property causes an element to behave as though it were an inline-level element, like a `<span>` or an `<a>` element. The style rules in the following example displays the `<p>` and `<li>` elements as inline-level elements:
    - `p { display: inline; } li { display: inline; }`

```
<!DOCTYPE html>
<html>
<head>
<style>
li {
  display: inline;
}
</style>
</head>
<body>

<p>Display a list of links as a horizontal menu:</p>

<ul>
  <li><a href="/html/default.asp" target="_blank">HTML</a></li>
  <li><a href="/css/default.asp" target="_blank">CSS</a></li>
  <li><a href="/js/default.asp" target="_blank">JavaScript</a></li>
</ul>
```

Display a list of links as a horizontal menu:

HTML CSS JavaScript

CSS Display

- **Display Inline-Block**
  - The inline-block value of the display property causes an element to generate a block box that will be flowed with surrounding content i.e. in the same line as adjacent content. The following style rules displays the `<div>` element as inline-block:
    - `div { display: inline-block; }`

```
<html lang="en">
<head>
<title>CSS Website Layout</title>
<style>
/* Style the top navigation bar */
ul {
    width:auto;
    list-style-type:none;
    background-color: black;
    color: white;
}
li {
    display:inline-block;
    padding: 14px 16px;
}
</style>
</head>
<nav>
  <ul>
    <li>Home</li>
    <li>News</li>
    <li>Contact</li>
    <li>About</li>
  </ul>
</nav>
```

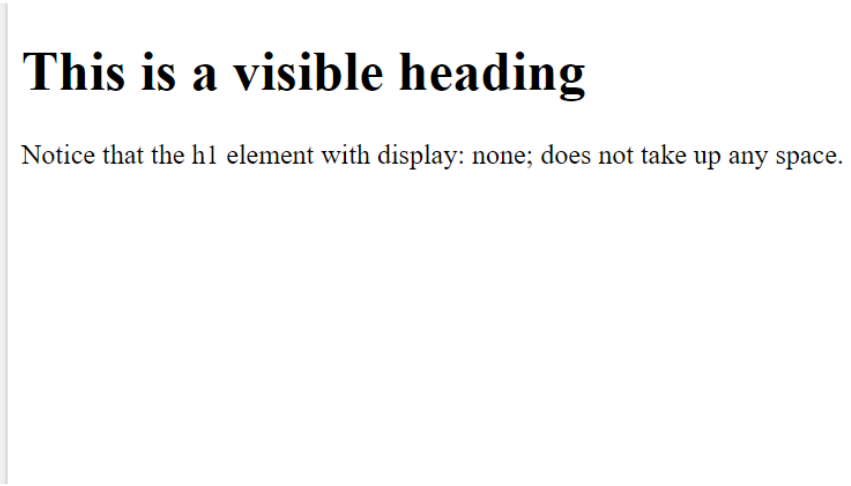


CSS Display

- **Display None**
  - The value none simply causes an element to generate no boxes at all. Child elements do not generate any boxes either, even if their display property is set to something other than none.
  - The document is rendered as though the element did not exist in the document tree.
    - `h1 { display: none; } p { display: none; }`
  - The value none for the display property does not create an invisible box — it creates no box at all.

```
<!DOCTYPE html>
<html>
<head>
<style>
h1.hidden {
  display: none;
}
</style>
</head>
<body>

<h1>This is a visible heading</h1>
<h1 class="hidden">This is a hidden heading</h1>
<p>Notice that the h1 element with display: none; does not take up any
space.</p>
```



CSS Float

- One way to position elements on a page is with the float property.
- The float property is versatile and can be used in a number of different ways.
- The float property allows us to take an element, remove it from the normal flow of a page, and position it to the left or right of its parent element.
- All other elements on the page will then flow around the floated element.
- When the float property is used on multiple elements at the same time, it provides the ability to create a layout by floating elements directly next to or opposite each other, as seen in multiple-column layouts.
- The two most popular values are **left** and **right**, which allow elements to be floated to the left or right of their parent element.
  - `img { float: right; }`

CSS Float

- The float property specifies how an element should float.
- <html>
  - **<img src="download.jpg" alt="sparrow" style="width:170px;height:170px;margin-left:15px;">**
  - ** In this example… **
- </html>
- <style>
  - **img { float: right; }**
- </style>

The float Property

In this example, the image will float to the right in the text, and the text in the paragraph will wrap around the image.



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui eget tellus gravida venenatis. Integer fringilla congue eros non fermentum. Sed dapibus pulvinar nibh tempor porta. Cras ac leo purus. Mauris quis diam velit. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui eget tellus gravida venenatis. Integer fringilla congue eros non fermentum. Sed dapibus pulvinar nibh tempor porta. Cras ac leo purus. Mauris quis diam velit. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui eget tellus gravida venenatis. Integer fringilla congue eros non fermentum. Sed dapibus pulvinar nibh tempor porta. Cras ac leo purus. Mauris quis diam velit. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui eget tellus gravida venenatis. Integer fringilla congue eros non fermentum. Sed dapibus pulvinar nibh tempor porta. Cras ac leo purus. Mauris quis diam velit.

CSS Position

- The position property identifies how an element is positioned on a page and whether or not it will appear within the normal flow of a document.
- It is used in conjunction with the box offset properties—`top`, `right`, `bottom`, and `left`—which identify exactly where an element will be positioned by moving elements in a number of different directions.
- By default every element has a position value of static, which means that it exists in the normal flow of a document and it doesn't accept any box offset properties.
- The static value is most commonly overwritten with a relative or absolute value.
- There are five different position values:
  - static
  - relative
  - fixed
  - absolute
  - Sticky

position: static

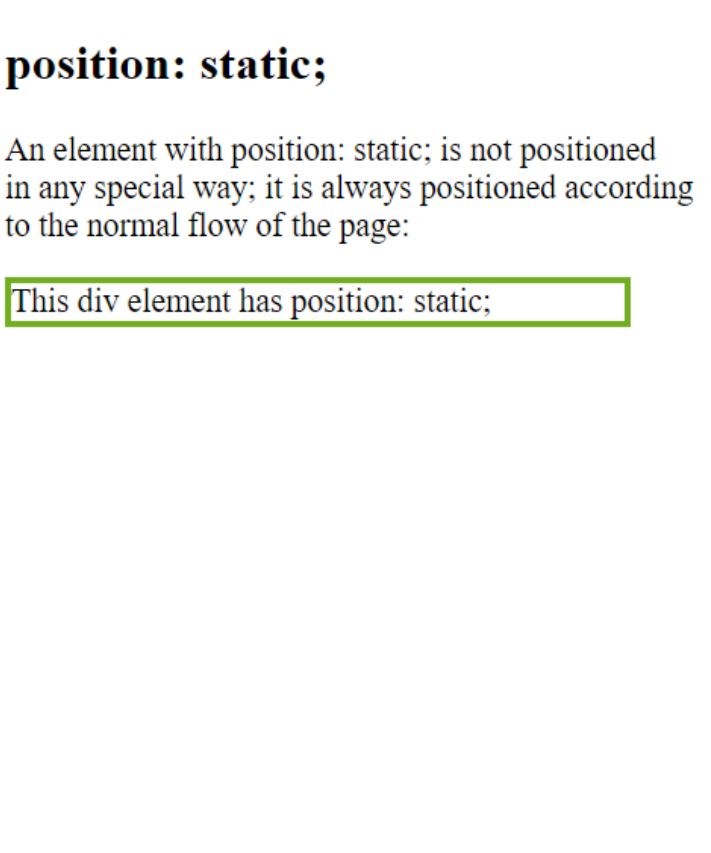
- An element with position: static is not positioned in any special way.
- It is always positioned according to the normal flow of the page.
- HTML elements are positioned static by default.
- Static positioned elements are not affected by the top, bottom, left, and right properties.

```
<!DOCTYPE html>
<html>
<head>
<style>
div.static {
  position: static;
  width : 300px;
  border: 3px solid #73AD21;
}
</style>
</head>
<body>

<h2>position: static;</h2>

<p>An element with position: static; is not
positioned <br> in any special way; it is
always positioned according <br> to the normal
flow of the page:</p>

<div class="static">
  This div element has position: static;
</div>
```



position: fixed

- An element with position: fixed; is positioned relative to the viewport, which means it always stays in the same place even if the page is scrolled.
- The top, right, bottom, and left properties are used to position the element. The value 0 indicates which side is used to fix the position.

```
<!DOCTYPE html>
<html>
<head>
<style>
div.fixed {
  position: fixed;
  top: 0;
  left: 0;
  width: 300px;
  border: 3px solid #73AD21;
}
</style>
</head>
<body>

<h2>position: fixed;</h2>

<p>An element with position: fixed; is positioned
relative <br> to the viewport, which means it
always stays in the same place <br> even if the
page is scrolled:</p>

<div class="fixed">
This div element has position: fixed;
</div>
```

This div element has position: fixed;

position: fixed;

An element with position: fixed; is positioned relative to the viewport, which means it always stays in the same place even if the page is scrolled:

position: relative

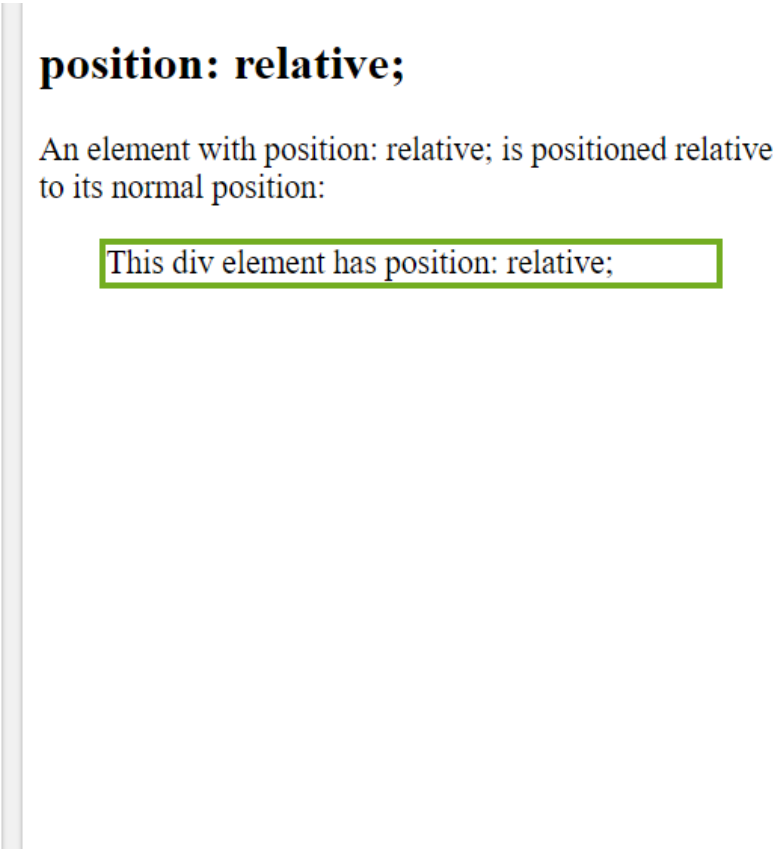
- An element with position: relative; is positioned relative to its normal position.
- Setting the top, right, bottom, and left properties of a relatively-positioned element will cause it to be adjusted away from its normal position. Other content will not be adjusted to fit into any gap left by the element.

```
<!DOCTYPE html>
<html>
<head>
<style>
div.relative {
  position: relative;
  left: 30px;
  width: 300px;
  border: 3px solid #73AD21;
}
</style>
</head>
<body>

<h2>position: relative;</h2>

<p>An element with position: relative; is
positioned relative <br> to its normal
position:</p>

<div class="relative">
This div element has position: relative;
</div>
```



position: absolute

- An element with position: absolute; is positioned relative to the nearest positioned ancestor (instead of positioned relative to the viewport, like fixed).
- If an absolute positioned element has no positioned ancestors, it uses the document body, and moves along with page scrolling.

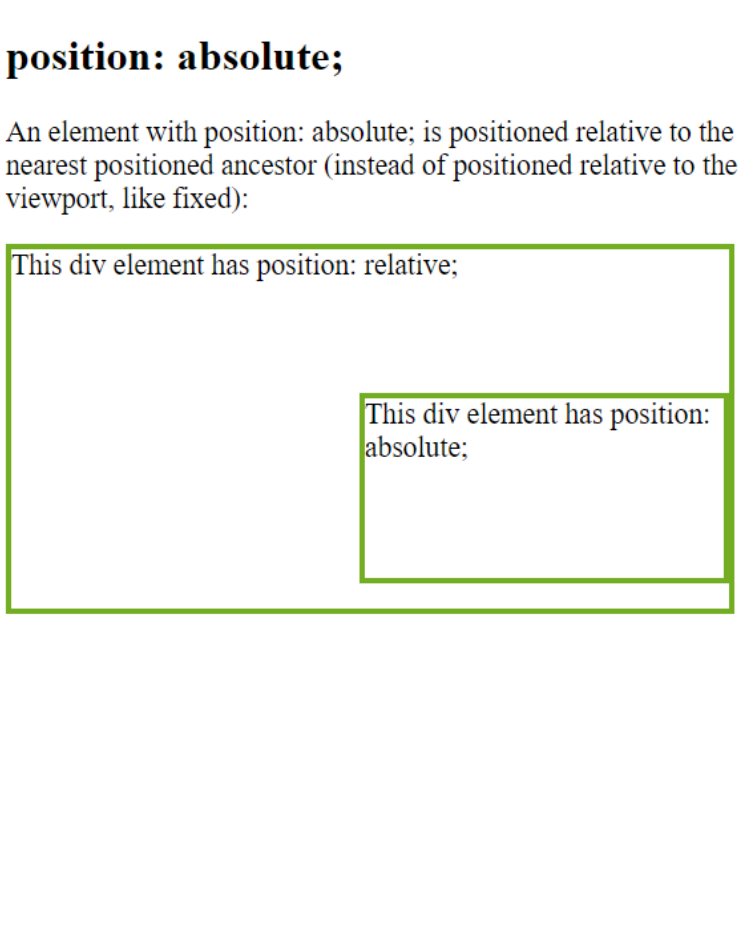
```
div.relative {
  position: relative;
  width: 400px;
  height: 200px;
  border: 3px solid #73AD21;
}

div.absolute {
  position: absolute;
  top: 80px;
  right: 0;
  width: 200px;
  height: 100px;
  border: 3px solid #73AD21;
}
</style>
</head>
<body>

<h2>position: absolute;</h2>

<p>An element with position: absolute; is positioned relative to
the <br> nearest positioned ancestor (instead of positioned
relative to the <br> viewport, like fixed):</p>

<div class="relative">This div element has position: relative;
  <div class="absolute">This div element has position: absolute;
</div>
</div>
```



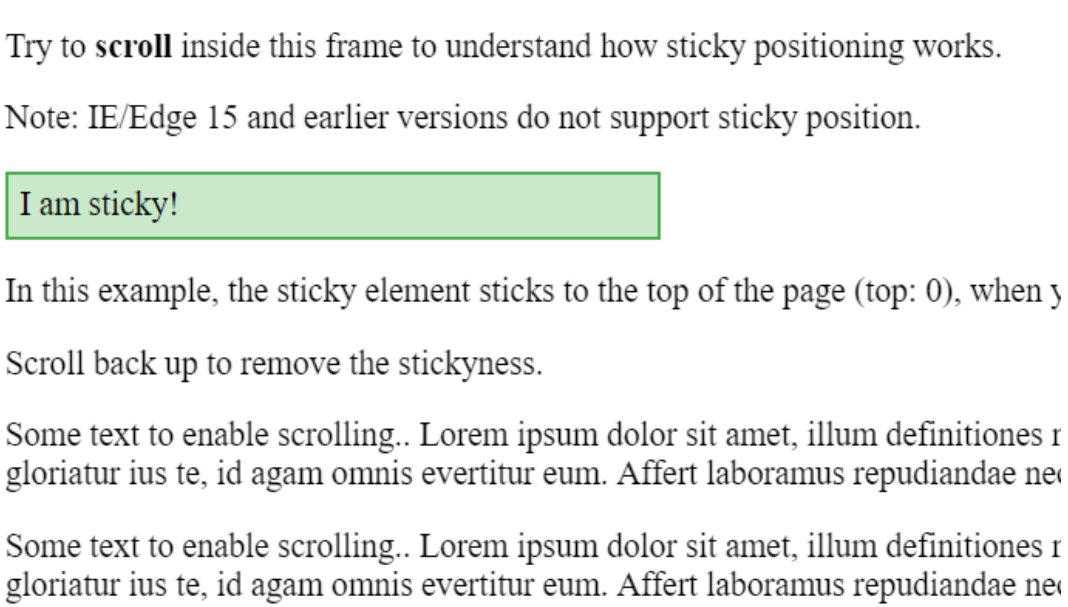
position: sticky

- An element with position: sticky; is positioned based on the user's scroll position.
- A sticky element toggles between relative and fixed, depending on the scroll position. It is positioned relative until a given offset position is met in the viewport - then it "sticks" in place (like position:fixed).

```
<!DOCTYPE html>
<html>
<head>
<style>
div.sticky {
  position: -webkit-sticky;
  position: sticky;
  top: 0;
  width: 300px;
  padding: 5px;
  background-color: #cae8ca;
  border: 2px solid #4CAF50;
}
</style>
</head>
<body>

<p>Try to <b>scroll</b> inside this
frame to understand how sticky
positioning works.</p>
<p>Note: IE/Edge 15 and earlier
versions do not support sticky
position.</p>

<div class="sticky">I am sticky!</div>
```



CSS Opacity

- The opacity property specifies the opacity/transparency of an element. it is now a part of the CSS3 specifications, but it was present for a long time. However, older browsers have different ways of controlling the opacity or transparency.
  - `img { opacity: 0.7; }`
  - The above style rule will make the paragraph element 70% opaque (or 30% transparent).
  - The opacity property takes a value a value from 0.0 to 1.0. A setting of opacity: 1; would make the element completely opaque (i.e. 0% transparent), whereas opacity: 0; would make the element completely transparent (i.e. 100% transparent).

The `opacity` property can take a value from 0.0 - 1.0. The lower value, the more transparent:



opacity 0.2 | opacity 0.5 | opacity 1 (default)

Activate

CSS Icons

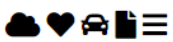
- The simplest way to add an icon to your HTML page, is with an icon library, such as Font Awesome.
- Add the name of the specified icon class to any inline HTML element (like <i> or).
- All the icons in the icon libraries below, are scalable vectors that can be customized with CSS (size, color, shadow, etc.)
- To use the Font Awesome icons, go to fontawesome.com, sign in, and get a code to add in the <head> section of your HTML page:
  - `<script src="https://kit.fontawesome.com/yourcode.js"></script>`

```
<!DOCTYPE html>
<html>
<head>
<title>Font Awesome Icons</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<script src="https://kit.fontawesome.com/a076d05399.js"></script>
<!--Get your own code at fontawesome.com-->
</head>
<body>

<p>Some Font Awesome icons:</p>
<i class="fas fa-cloud"></i>
<i class="fas fa-heart"></i>
<i class="fas fa-car"></i>
<i class="fas fa-file"></i>
<i class="fas fa-bars"></i>

<p>Styled Font Awesome icons (size and color):</p>
<i class="fas fa-cloud" style="font-size:24px;"></i>
<i class="fas fa-cloud" style="font-size:36px;"></i>
<i class="fas fa-cloud" style="font-size:48px;color:red;"></i>
<i class="fas fa-cloud" style="font-size:60px;color:lightblue;"></i>
```

Some Font Awesome icons:



Styled Font Awesome icons (size and color):



Summary of Today's Lecture

- CSS Box Model
  - Height and Width
  - Padding
  - Borders
  - Margins
- CSS Box Sizing
- CSS Text
- CSS Fonts
- CSS Colors
- CSS Backgrounds
- CSS Links
- CSS Lists
- CSS Tables
- CSS Display
- CSS Position
- CSS Opacity
- CSS Icons